
Connection-Aware Autoscaling for Stateful WebSocket Workloads in Kubernetes: Design, Evaluation, and Empirical Validation of a Custom Controller

Abhash Behera · Aditya Hota · Suprit Kumar Naik · Ritesh Kumar Singh · Rakesh Ranjan Kumar

Abstract Kubernetes Horizontal Pod Autoscaler (HPA) scales containerized workloads based on CPU utilization, a strategy that works well for stateless, request-driven microservices. However, modern cloud-native architectures increasingly rely on **stateful, persistent-connection protocols** such as WebSocket, where workload state resides in long-lived network connections rather than transient CPU consumption. This paper presents a systematic empirical investigation demonstrating that CPU-based HPA is fundamentally unsuitable for WebSocket workloads. Through a progressive chain of five experiments, we show that HPA (i) over-provisions under cyclic load, hitting `maxReplicas` within 99 seconds; (ii) induces reconnection storms of up to 1,400 connections per second during scale-down events; and (iii) permanently destroys live idle connections in a staircase pattern synchronized with scale-down decisions. To address these failures, we design and implement a custom Kubernetes controller - the `StatefulAutoscaler` - that scales exclusively on `sum(active_connections)` queried from Prometheus

and incorporates a sliding-window stabilization mechanism to prevent premature scale-down during transient connection drops. Experimental evaluation demonstrates that the custom controller scales to exactly `ceil(connections/target)` replicas with zero over-provisioning, holds all pods warm through a 90-second connection gap via its stabilization window, and achieves zero connection loss during any scaling event. These results validate that connection-aware autoscaling with restart stabilization is both necessary and sufficient for managing stateful WebSocket deployments in Kubernetes.

Keywords Kubernetes · Horizontal Pod Autoscaler · WebSocket · Stateful Workloads · Custom Controller · Connection-Aware Scaling · Prometheus

1 Introduction

Kubernetes has become the de facto orchestration platform for containerized applications, providing declarative workload management, service discovery, and automated scaling [1, 13]. A central component of its elasticity model is the **Horizontal Pod Autoscaler (HPA)**, which dynamically adjusts replica counts based on observed resource metrics - predominantly CPU utilization [6]. For stateless, request-driven workloads (e.g., HTTP APIs), where CPU consumption correlates directly with incoming request volume, this proportional control strategy performs predictably and converges reliably [2].

However, the landscape of modern cloud-native applications is shifting toward **stateful, persistent-connection protocols**. Real-time collaboration tools, IoT telemetry pipelines, financial trading platforms, multiplayer games, and chat systems increasingly rely on WebSocket [4] - a full-duplex, long-lived TCP protocol where connections persist for minutes to hours. In such

Abhash Behera
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: abhas@placeholder.edu

Aditya Hota
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India

Suprit Kumar Naik
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India

Ritesh Kumar Singh
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India

Rakesh Ranjan Kumar (Supervisor)
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: rakeshranjan@cgu-odisha.ac.in

workloads, the fundamental unit of “state” is not a CPU cycle or an HTTP request, but an *active network connection* maintained between client and server.

This distinction introduces a critical architectural mismatch: **CPU utilization does not encode connection state**. A WebSocket server holding 800 idle connections may consume near-zero CPU, yet terminating a single pod destroys all connections it hosts, forcing clients into reconnection storms. Despite this mismatch, Kubernetes provides no built-in mechanism for connection-aware scaling, and the default HPA remains the recommended autoscaling solution for all workload types.

This paper makes three contributions:

1. **Empirical evidence chain.** Through four progressive experiments (A, B1, B2-Instrumented, B3), we systematically demonstrate that CPU-based HPA fails for WebSocket workloads - from over-provisioning under cyclic load, through quantified reconnection storms ($\sim 1,400$ conn/s), to permanent staircase destruction of live idle connections.
2. **Custom controller design.** We present the `StatefulAutoscaler`, a Kubernetes custom controller built with the `Kubebuilder` [7] SDK that scales exclusively on `sum(active_connections)` and implements a sliding-window stabilization mechanism (`ScaleDownCooldownSeconds`) to absorb transient connection drops without premature scale-down.
3. **Experimental validation.** We validate the custom controller (Experiment C) under a two-cycle restorm simulation, demonstrating zero connection loss, exact replica targeting ($\text{ceil}(800/100) = 8$), and successful pod preservation through a 90-second connection gap.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 describes the Kubernetes autoscaling pipelines evaluated as baselines. Section 4 presents the progressive failure evidence. Section 5 details the custom controller design. Section 6 presents experimental results. Section 7 discusses implications. Section 8 addresses limitations and future work. Section 9 concludes.

2 Related Work

2.1 Kubernetes Autoscaling

The Kubernetes HPA implements a discrete-time proportional controller that evaluates metrics at a fixed synchronization interval (default: 15 seconds) and computes desired replicas using:

$$\text{desired} = \left\lceil \text{current} \times \frac{\text{currentMetric}}{\text{targetMetric}} \right\rceil \quad (1)$$

This formulation produces stepwise, reactive scaling extensively studied for stateless workloads [2, 9]. Casalicchio and Perciballi [2] showed that metric selection directly influences scaling stability, but their work focused exclusively on CPU/memory metrics for HTTP workloads.

2.2 Custom and Application-Level Metrics

Kubernetes supports custom metrics via the Prometheus Adapter [3], enabling HPA to scale on application-defined signals such as request rate or queue depth. Prior work has explored HTTP-rate-based scaling [10] and hybrid CPU+traffic strategies [11]. However, these approaches target request-response protocols and do not address persistent-connection workloads where connections survive across request boundaries.

2.3 Machine Learning and Predictive Scaling

Recent work has applied reinforcement learning [11, 5] and machine learning [12, 15] to container autoscaling. While promising, these approaches introduce model training complexity, lack interpretability, and have not been validated against stateful WebSocket workloads.

2.4 Gap Addressed

To the best of our knowledge, no prior work has (i) empirically quantified the failure modes of CPU-based HPA for persistent WebSocket connections, (ii) measured reconnection storm magnitude as a function of scale-down events, or (iii) designed and validated a connection-aware Kubernetes controller with stabilization semantics specifically targeting restorm resilience. This paper addresses all three gaps.

3 Background: Kubernetes Autoscaling Pipelines

Before evaluating WebSocket-specific failures, we characterize the baseline behavior of HPA under two metric pipelines. These preliminary experiments inform the design of our custom controller.

3.1 Native Kubernetes Resource Metrics (KRM)

In the KRM pipeline, CPU usage flows from kubelets (via `cAdvisor`) through the Metrics Server to the `metrics.k8s.io` API, where HPA consumes it. Our KRM baseline used a CPU-intensive HTTP workload with square-wave load cycles (100s HIGH at 50 req/s, 100s LOW at 2 req/s) on a 4-node Kind cluster [8]. Key findings:

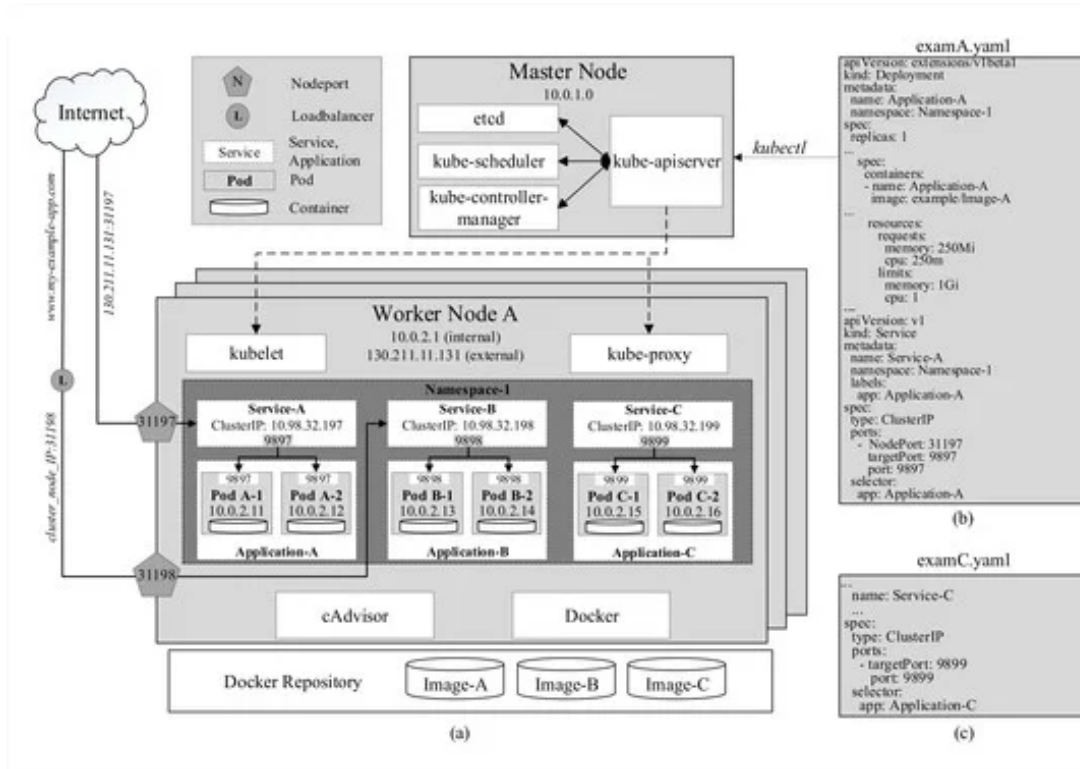


Fig. 1 Standard Kubernetes HPA feedback loop architecture (source: [14]). The HPA controller periodically reads CPU metrics from the API server (via Metrics Server or Prometheus Adapter) and computes desired replica counts proportionally. This architecture assumes the observed metric accurately reflects workload pressure - an assumption that fails for persistent-connection servers.

- **Staircase scaling:** Replicas increase in discrete steps aligned with the HPA sync cycle (Figure 2).
- **CPU overshoot:** Utilization exceeds 200% of requested CPU at burst onset.
- **Actuation delay:** Measurable divergence between desiredReplicas and currentReplicas from scheduling and startup (Figure 3).
- **Stable convergence:** No oscillatory instability despite transient deviations.

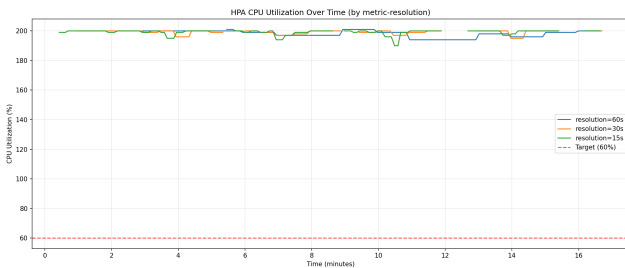


Fig. 2 KRM baseline: CPU utilization over time showing overshoot above the 60% target during burst onset, with gradual convergence as replicas increase.

3.2 Prometheus Custom Metrics (PCM)

The PCM pipeline introduces additional stages: pods expose metrics to Prometheus, the Prometheus Adapter transforms them via PromQL, and serves them through the Custom Metrics API to HPA. We evaluated PCM-CPU, PCM-H (HTTP rate), and PCM-CH (hybrid) across scrape intervals of 60s, 30s, and 15s. Key findings:

- **Scrape interval sensitivity:** Reducing from 60s to 30s significantly improved scale-up latency; the gain from 30s to 15s was marginal (Figure 4).
- **HTTP as leading indicator:** PCM-H reacted earliest to traffic bursts but caused temporary over-provisioning.
- **Hybrid stability:** PCM-CH combined early detection with stabilizing CPU feedback.

These baselines confirm that HPA behaves as a stable proportional controller for *stateless* workloads. The critical question is: **what happens when CPU ceases to correlate with workload state?**

3.3 The HPA Architecture

Figure 1 (top of page) illustrates the standard Kubernetes HPA feedback loop as described by Yin et al. [14]. The

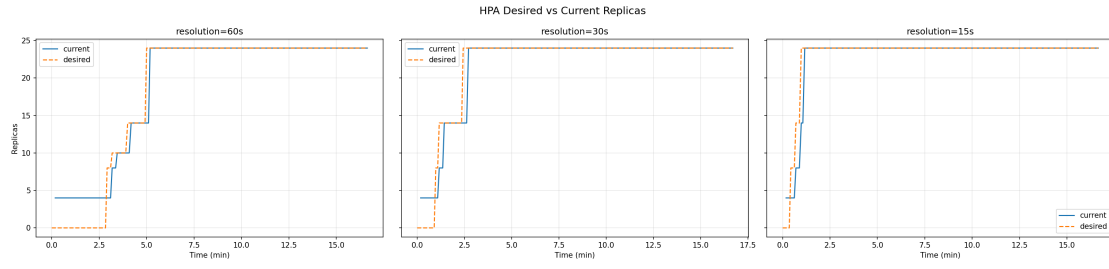


Fig. 3 KRM baseline: Desired vs. current replica divergence during scale-up events, quantifying actuation delay from pod scheduling and container startup.

PCM-CPU: Scraping Period Comparison

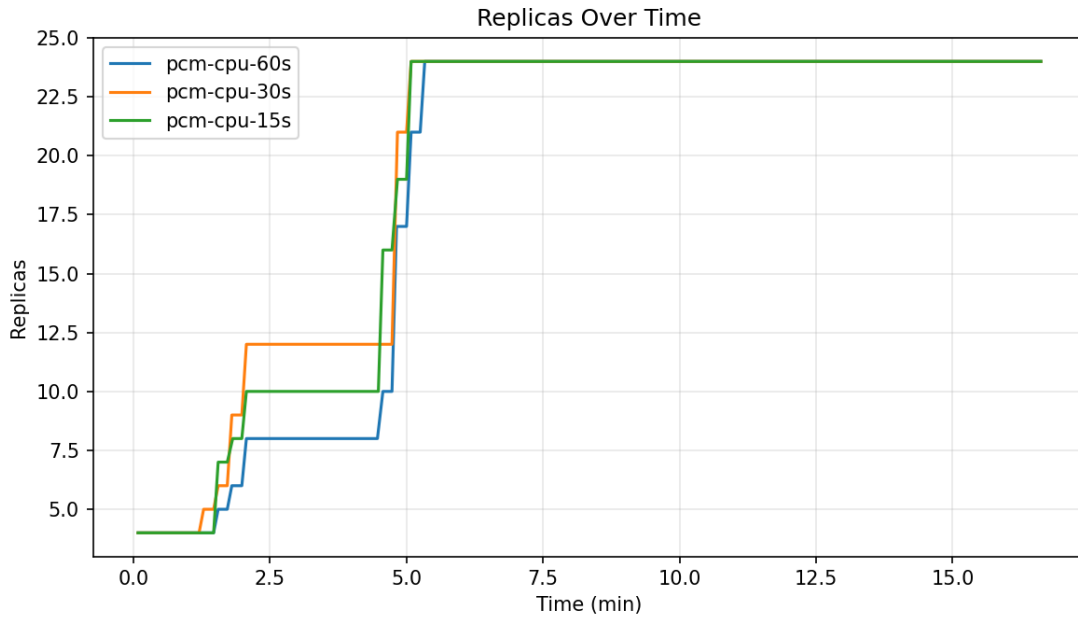


Fig. 4 PCM baseline: Impact of Prometheus scrape interval on scaling responsiveness. The 60s interval causes visible staircase delay; 15s aligns metric updates with the HPA sync cycle.

controller periodically reads metrics from the API server and adjusts replica counts proportionally. This architecture assumes that the observed metric accurately reflects workload pressure - an assumption that fails for connection-holding servers.

4 Problem: HPA Failure for WebSocket Workloads

This section presents a progressive chain of four experiments that systematically expose the failure of CPU-based HPA when applied to persistent WebSocket workloads. All experiments use a Kind cluster [8] with a custom WebSocket server, 800 persistent client connections, and HPA configured with `targetCPUUtilization: 60%`.

4.1 Experiment A: HPA Baseline Under Monotonic Load

Hypothesis: Under steady, monotonic WebSocket load where every connection generates CPU work (`CPU_WORK=1`), HPA will scale correctly.

Setup: 800 WebSocket clients connect over 300 seconds. Each received ping triggers a CPU spin loop, ensuring $\text{CPU} \propto \text{connections}$. HPA: `minReplicas=2`, `maxReplicas=10`, `target CPU 60%`.

Results: HPA scaled smoothly from 2 to 5 replicas as CPU rose proportionally with connections. After load removal, the system respected the 300-second default stabilization window before scaling down. No oscillation, no reconnection storms observed.

The replica timeline (Figure 5) confirms ideal HPA behavior: stepwise scale-up aligned with CPU, a flat plateau enforced by the stabilization window, and a clean

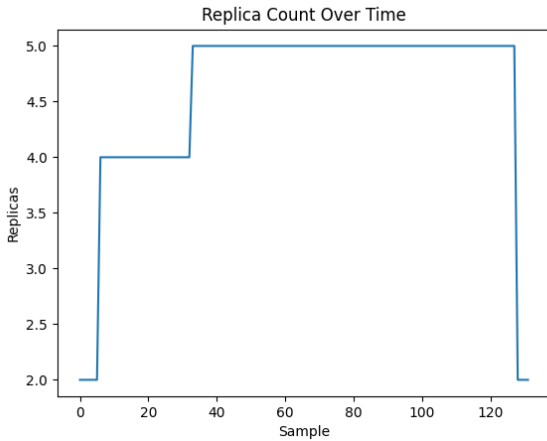


Fig. 5 Experiment A: Clean, monotonic HPA scaling under steady WebSocket load. Staircase rise (2→5) followed by the stabilization window plateau and clean descent to 2 - the ideal baseline behavior.

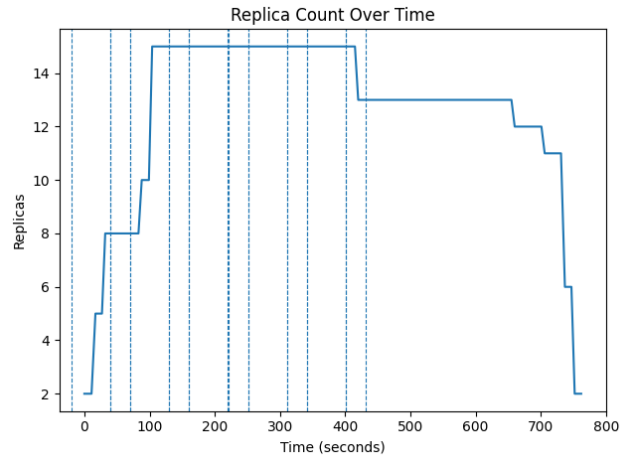


Fig. 6 Experiment B1: HPA saturates at `maxReplicas=15` within 99s under cyclic load. Scale-down takes ~650s. The system oscillates between its replica ceiling and floor across cycles.

return to minimum. This is the only scenario in which HPA succeeds.

Conclusion: HPA works correctly *only* when CPU perfectly correlates with connection count - an idealized condition rarely met in production.

4.2 Experiment B1: HPA Under Cyclic Load

Hypothesis: Under cyclic HIGH/LOW load patterns, HPA will over-provision and exhibit slow recovery.

Setup: Alternating HIGH phases (800 connections, active pinging) and LOW phases (connections held idle, no pinging). `maxReplicas` increased to 15. Scale-down stabilization reduced to 60 seconds.

Results: HPA hit `maxReplicas=15` within 99 seconds during HIGH phases - 87.5% more pods than theoretically needed. Scale-down during LOW phases took approximately 650 seconds. The replica timeline (Figure 6) shows the system oscillating between ceiling and floor.

Conclusion: Cyclic workloads cause HPA to saturate at `maxReplicas` while recovering disproportionately slowly - wasting resources in both directions.

4.3 Experiment B2-Instrumented: Quantifying Reconnection Storms

Hypothesis: Each HPA scale-down event will trigger measurable reconnection storms exceeding 1,000 connections/second, with connection overshoot above the 800-connection target.

Setup: The WebSocket server was instrumented with two Prometheus metrics: `active_connections` (gauge) and `new_connections_total` (counter). Prometheus

scraped every 15 seconds. Five HIGH/LOW cycles executed.

Results: Every scale-down event triggered a reconnection storm. The peak reconnection rate reached ~1,400 conn/s (Figure 7). Connection count overshoot from 800 to 1,215 (51.9% above target, Figure 8) as new connections arrived before old connection state was cleaned up.

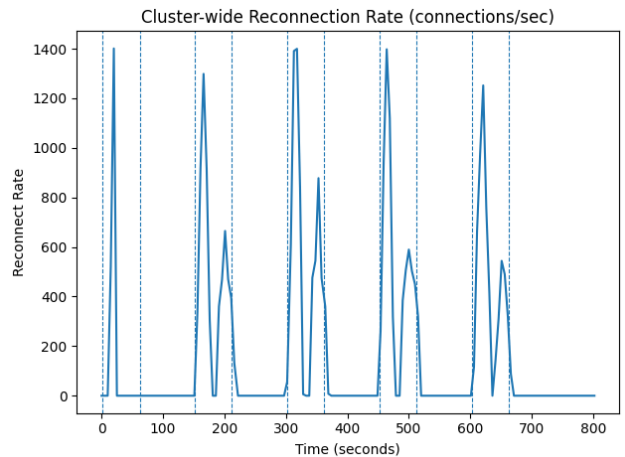


Fig. 7 Experiment B2-Instrumented: Reconnection rate (conn/s) over time. Each spike corresponds to an HPA scale-down event, with peak rates exceeding 1,400 conn/s - a direct measure of client disruption.

Figure 9 correlates the replica oscillation directly with the resulting connection instability, showing the causal chain from scale-down decision to client-side disruption.

Conclusion: HPA scale-down events are directly destructive to WebSocket clients, triggering high-rate

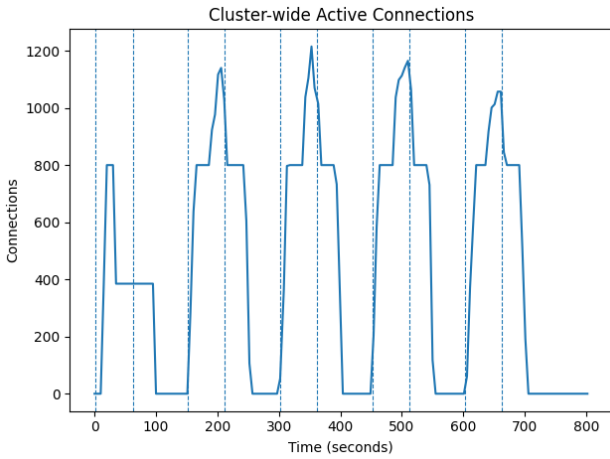


Fig. 8 Experiment B2-Instrumented: Active connection count showing overshoot to 1,215 - 51.9% above the 800-connection target - during reconnection storms triggered by HPA scale-down events.

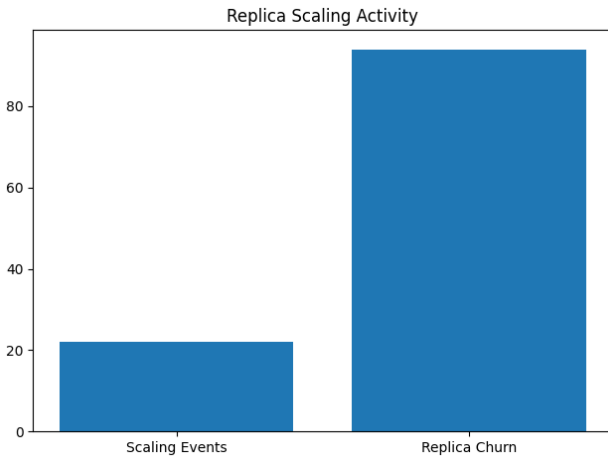


Fig. 9 Experiment B2-Instrumented: Overlay of HPA replica changes and active connection count. Each downward step in replicas triggers a corresponding spike in connection disruption.

reconnection storms that cause temporary over-provisioning and degrade user experience.

4.4 Experiment B3: Permanent Connection Loss

Hypothesis: If WebSocket clients do *not* reconnect after displacement (simulating clients that abandon or fail), HPA scale-down events will cause permanent, irrecoverable connection loss.

Setup: Same as B2-Instrumented, but the load generator was configured with `reconnect: false`. Once a client's connection is severed, it does not attempt to reconnect. `CPU_WORK=1` keeps HPA operational; connections are the primary metric of interest.

Results: The connection graph (Figure 10) shows a distinctive **staircase step-down** pattern. Each step

corresponds precisely to an HPA scale-down event: when a pod is terminated, all its connections are permanently lost.



Fig. 10 Experiment B3: Staircase permanent destruction of active connections. Each step-down in the connection count (800→744→697→445→79) is precisely synchronized with an HPA scale-down event.

The combined view (Figure 11) provides the definitive causal proof: CPU drops to near-zero because connections become idle, which *causes* HPA to scale down, which *permanently destroys* those connections. This is HPA's fundamental architectural incompatibility with stateful workloads.

Conclusion: HPA's reliance on CPU actively destroys the workload state it should be protecting. This is not a configuration problem; it is an architectural failure.

4.5 Summary of HPA Failure Evidence

Table 1 summarizes the progressive evidence chain:

5 Proposed Solution: The StatefulAutoscaler Controller

Based on the failure evidence, we designed and implemented a custom Kubernetes controller that addresses HPA's three fundamental shortcomings: wrong scaling signal, wrong scale-down semantics, and no restart protection.

5.1 Design Principles

The StatefulAutoscaler controller is built on three architectural principles:

1. **Scale on the right signal.** Query `sum(active.connections)` from Prometheus - the metric that directly encodes workload state.
2. **Compute replicas from connection density.** Given a target threshold T connections per pod:

$$desired = \left\lceil \frac{\text{sum(active.connections)}}{T} \right\rceil \quad (2)$$

This produces exact, proportional scaling with zero CPU dependency.

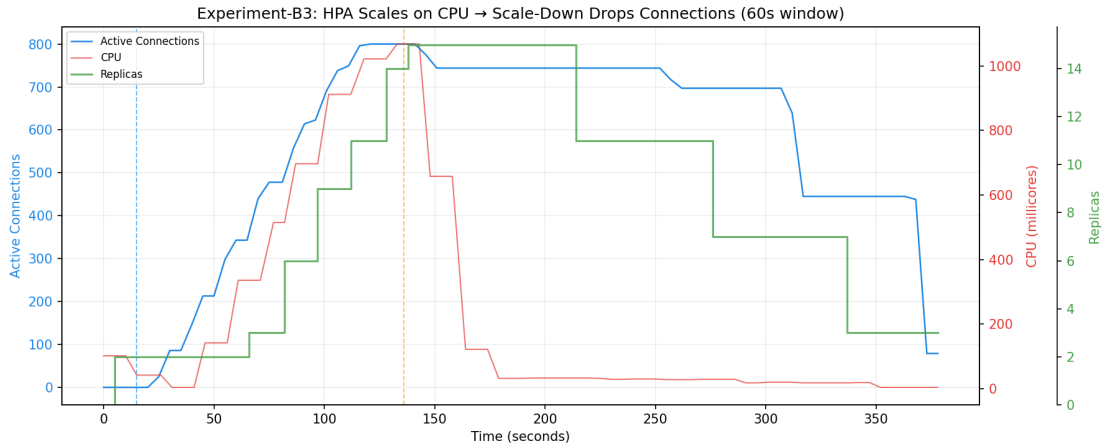


Fig. 11 Experiment B3 combined view - the causal proof of HPA's architectural failure: CPU (red) drops near zero (idle connections generate no CPU) → HPA scales down replicas (green) → active connections (blue) are permanently destroyed. The autoscaler destroys the very state it should protect.

Table 1 Progressive evidence chain demonstrating HPA failure for WebSocket workloads.

Experiment	Condition	HPA Behavior	Key Finding
A	Monotonic, CPU $\propto$ connections	Correct scaling	Works only under ideal CPU correlation
B1	Cyclic HIGH/LOW load	maxReplicas in 99s	87.5% over-provisioning, 650s recovery
B2-Inst	Cyclic + instrumented	Reconnection storms	Peak 1,400 conn/s; 51.9% overshoot
B3	No-reconnect clients	Permanent loss	Staircase destruction of live connections

3. **Stabilize scale-down with connection history.** A sliding-window mechanism holds the maximum desired replica count observed in the last N seconds, preventing premature scale-down during transient connection drops.

5.2 Custom Resource Definition

The controller introduces a CRD: `StatefulAutoscaler` (API group: `autoscaling.star.local/v1alpha1`). Table 2 summarizes the key spec fields.

Table 2 `StatefulAutoscaler` CRD specification fields.

Field	Purpose
<code>targetRef</code>	Deployment to scale
<code>minReplicas</code>	Floor replica count
<code>maxReplicas</code>	Ceiling replica count
<code>targetConnsPerPod</code>	Connection density target T
<code>maxScaleUpStep</code>	Max pods to add per cycle
<code>maxScaleDownStep</code>	Max pods to remove per cycle
<code>scaleDownCooldownSec</code>	Stabilization window duration (s)

5.3 Reconciliation Loop

The controller runs a reconciliation loop every 5 seconds. Algorithm 1 describes the core logic, with particular attention to the stabilization window (lines 6–8).

Algorithm 1 `StatefulAutoscaler` Reconciliation

```

Input: StatefulAutoscaler CR  $S$ , Deployment  $D$ 
1:  $C \leftarrow \text{QUERYPROMETHEUS}(\text{sum}(\text{active\_connections}))$ 
2: if  $C$  is unavailable then
3:   request after 10s ▷ Safe default: no action
4: end if
5:  $raw \leftarrow \lceil C/S.\text{targetConnsPerPod} \rceil$ 
6:  $raw \leftarrow \max(S.\text{min}, \min(S.\text{max}, raw))$ 
7:  $W.\text{append}(now, raw)$ 
8:  $W.\text{evict}(\text{entries older than } S.\text{cooldown})$ 
9:  $stabilized \leftarrow \max(W)$  ▷ Window max prevents premature scale-down
10:  $current \leftarrow D.\text{spec.replicas}$ 
11: if  $stabilized > current$  then
12:    $target \leftarrow \min(current + S.\text{maxScaleUpStep}, stabilized)$ 
13: else if  $stabilized < current$  then
14:    $target \leftarrow \max(current - S.\text{maxScaleDownStep}, stabilized)$ 
15: else
16:    $target \leftarrow current$ 
17: end if
18:  $\text{PATCHDEPLOYMENT}(D, target)$ 
19: request after 5s

```


5.4 The Stabilization Window

Without stabilization, if all 800 clients disconnect temporarily (network glitch, rolling update), the controller computes $\lceil 0/100 \rceil = 0 \rightarrow \text{minReplicas}=2$ and scales down - recreating the cascade HPA exhibited.

With a 120s stabilization window, the controller maintains $\max(W) = 8$ even when connections are zero. All pods are held warm. When clients reconnect within the window, they land on already-provisioned pods seamlessly.

This addresses a problem HPA cannot solve at any parameter configuration. HPA’s stabilization window operates on CPU recommendations. Even with a 300-second CPU window, HPA scales down idle-but-connected pods because CPU reads persistently near zero.

6 Experimental Evaluation: Experiment C

6.1 Setup

Experiment C maintains maximal comparability with B3 while replacing HPA with the StatefulAutoscaler. Table 3 highlights the key differences.

Table 3 Experiment C vs. B3 configuration comparison.

Dimension	B3 (HPA)	C (Custom)
Server image	ws-instrumented	ws-instrumented
CPU_WORK	1	0
Connections	800	800
Autoscaler	HPA (CPU)	StatefulAutoscaler
Scale-down guard	60s CPU window	120s connection window

Setting CPU_WORK=0 proves the controller needs no CPU signal while scaling purely on connection count - the most challenging case for any CPU-based system. The StatefulAutoscaler was configured with targetConnectionsPerPod: 100, maxScaleUpStep: 3, maxScaleDownStep: 2, and scaleDownCooldownSeconds: 120, giving an expected desired replica count of $\lceil 800/100 \rceil = 8$.

6.1.1 Load Pattern: Two-Cycle Restorm Simulation

- **CYCLE 1** (0–150s): 800 clients ramp up; controller scales to 8.
- **DROP 1** (150–240s): All clients deleted; connections $\rightarrow$ 0. *Critical test:* stabilization window must hold replicas at 8 for the full 90-second gap.
- **CYCLE 2** (240–390s): 800 new clients connect to already-warm pods.

- **FINAL DROP** (390–570s): Permanent disconnection; window expires; controller scales to 2.

6.2 CYCLE 1: Connection-Driven Scale-Up

The controller scaled monotonically from 2 to 8 as connections ramped to 800: the trajectory was $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$, completing within ~ 60 seconds. CPU remained below 176m total throughout - near zero, confirming complete CPU independence.

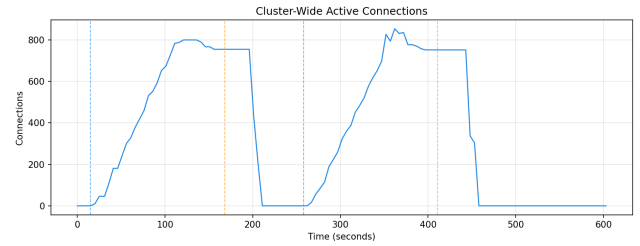


Fig. 12 Experiment C: Active connections over time. Two distinct blocks (CYCLE 1 and CYCLE 2) are separated by the 90-second DROP 1 gap. CYCLE 2 clients connect instantly to already-warm pods.

Figure 12 shows the two-block connection profile: CYCLE 1 rises to 800, drops to zero during DROP 1, then CYCLE 2 rises again quickly - clients connecting to pre-provisioned pods with near-zero latency.

6.3 DROP 1: Stabilization Window Proof

At $t \approx 200$ s, all clients were deleted and connections dropped to zero within 15 seconds. The stabilization window held the replica count **perfectly flat at 8** throughout the 90-second gap. The logical trace:

- $\text{rawDesired} = \lceil 0/100 \rceil = 0 \rightarrow$ clamped to $\text{minReplicas}=2$
- Window contained entries from prior 120s showing $\text{desired} = 8$
- $\text{stabilizedDesired} = \max(W) = 8$
- Replicas held at 8 - zero scale-down actions taken

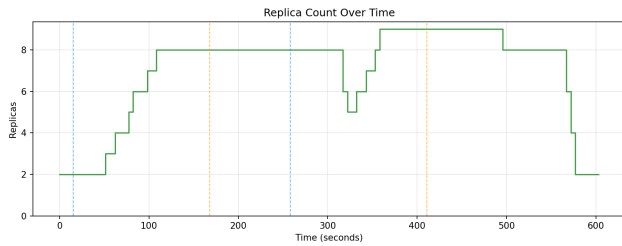
Figure 13 shows the characteristic “flat bridge” at 8 replicas across the DROP 1 gap - a pattern entirely absent from every HPA experiment. At 60 seconds into the gap, B3’s HPA had already destroyed 56 connections; the custom controller held every pod.

6.4 CYCLE 2 and Final Drop

At $t \approx 270$ s, 800 new clients connected to the 8 already-provisioned pods with zero latency. Connections

Table 4 Head-to-head: CPU-based HPA (Experiment B3) vs. Custom StatefulAutoscaler (Experiment C).

Metric	HPA (B3)	Custom Controller (C)
Scaling signal	CPU utilization (%)	<code>sum(active_connections)</code>
Replicas for 800 connections	15 (87.5% over-provisioned)	8 (exact: $\lceil 800/100 \rceil$)
Connections lost during scale-down	800→79 (staircase)	0
Pods held through 90s gap	0 (all scaled down)	8 (flat bridge)
Reconnection storms	Destroyed (B2: 1,400 conn/s peak)	None
Restorm recovery	Full re-scale required	Instant (warm pods)
CPU dependency	100%	0% (CPU_WORK=0)
Total scale events	7+ (oscillation)	4 (monotonic)
Peak CPU (total)	>1,200m	<176m

**Fig. 13** Experiment C: Replica count over time. The “flat bridge” at 8 replicas during DROP 1 (connections = 0, CPU = 0, replicas = 8) is the primary evidence that the stabilization window functions correctly. The final controlled descent (9→2) occurs only after the window fully expires.

peaked at 854, and replicas responded smoothly. No reconnection storm, no under-provisioning. After permanent disconnection at $t \approx 450$ s, the controller waited 120 seconds for the window to expire, then descended cleanly: 9→8→6→4→2. Zero connections were lost.

6.5 Combined View and Head-to-Head Comparison

Figure 14 overlays all three signals and provides the definitive visual contrast to Figure 11: connections drive replicas; CPU is irrelevant.

Table 4 quantifies the head-to-head outcome:

7 Discussion

7.1 CPU is the Wrong Signal

Experiments B3 and C collectively prove that CPU utilization is an unreliable proxy for workload health in connection-holding applications. In B3, CPU dropped to near-zero while 800 connections remained alive, causing HPA to terminate pods and destroy those connections. In Experiment C, the custom controller operated at CPU_WORK=0 - near-zero CPU throughout - yet scaled correctly to 8 replicas based purely on connection count. The scaling signal, not the algorithm, is the fundamental problem.

7.2 Stabilization Window Philosophy

The sliding-window mechanism represents a philosophically different approach from HPA’s stabilization. HPA stabilizes the *metric signal* (CPU), while the StatefulAutoscaler stabilizes *scaling intent* (the desired replica count derived from connections). Even if HPA’s CPU stabilization window is set to 300 seconds, it stabilizes a zero-CPU reading - still recommending scale-down. The custom controller’s window preserves a non-zero connection-derived desired replica count, correctly holding pods warm.

7.3 Step-Size Controls

The `maxScaleUpStep=3` and `maxScaleDownStep=2` parameters produced smooth, monotonic trajectories. HPA’s default 100% Percent policy produced jumps of 2→6→12→15 within seconds. Step-size limits prevent both ramp-up overshoot and violent scale-down contraction.

7.4 Generalizability

While validated on WebSocket, the StatefulAutoscaler design generalizes to any protocol where workload state is a persistent connection: gRPC streaming, MQTT broker connections, database connection pools, and SSE endpoints. The only requirement is a Prometheus gauge counting active connections.

8 Limitations and Future Work

- No graceful drain.** The drain field exists in the CRD (enabled: false). Production use requires migrating connections off a pod before termination.
- Single Prometheus instance.** Prometheus failure causes the controller to requeue with no scaling action. A production deployment needs metric source failover.

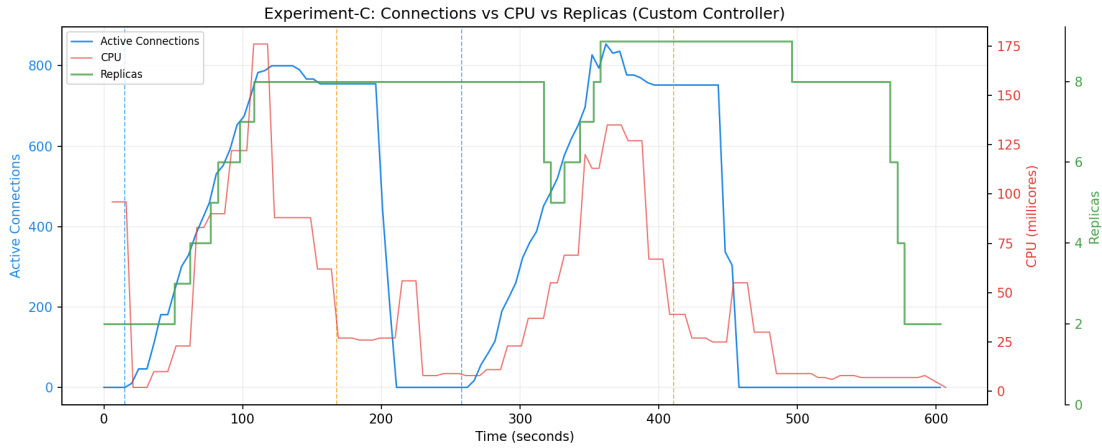


Fig. 14 Experiment C combined view: Connections (blue) drive replicas (green) through the custom controller’s logic. CPU (red) is flat near zero throughout - it never influences scaling decisions. The flat bridge in replicas during DROP 1 ($\approx 160\text{--}260\text{s}$) is the definitive proof that the stabilization window correctly prevents premature scale-down even when both connections and CPU read zero.

3. **In-memory stabilization history.** The sliding window is stored in process memory; controller restart resets history, potentially triggering premature scale-down.
4. **No connection distribution control.** The controller sets the global replica count but cannot rebalance connections across pods. Uneven distribution may yield suboptimal decisions.
5. **Sliding-window edge case.** During CYCLE 2, a brief replica dip to 5–6 occurred as entries aged out before new connections refilled the window. A minimum hold-time after the last scale-up would prevent this.

Planned enhancements include: graceful drain via a /drain endpoint, per-pod metric queries instead of cluster sum(), rate-based proactive scaling using new_connections_total, and persistent stabilization state stored in the CR status field.

9 Conclusion

This paper presented a systematic empirical investigation demonstrating that Kubernetes CPU-based HPA is architecturally incompatible with stateful WebSocket workloads. Through four progressive experiments, we proved that HPA over-provisions by 87.5% under cyclic load, induces reconnection storms of $\sim 1,400$ conn/s during scale-down, and permanently destroys live idle connections in a staircase pattern synchronized with scale-down events.

To address these failures, we designed and validated the StatefulAutoscaler - a custom Kubernetes controller that scales exclusively on `sum(active_connections)` with a sliding-window stabilization mechanism. Experiment C demonstrated:

- **Exact scaling:** 8 replicas for 800 connections (0% waste vs. HPA’s 87.5%)

- **Zero connection loss:** across all scaling events
- **Restorm survival:** 8 pods held warm through a 90-second full disconnection
- **Complete CPU independence:** controller operated at `CPU_WORK=0`

These results validate that connection-aware autoscaling with stabilization semantics is both necessary and sufficient for managing stateful WebSocket deployments in Kubernetes. The StatefulAutoscaler offers a practical, production-oriented solution for the growing class of cloud-native applications whose state resides in persistent network connections rather than transient CPU consumption.

References

1. Burns B, Grant B, Oppenheimer D, Brewer E, Wilkes J (2016) Borg, omega, and kubernetes. ACM Queue 14(1):70–93
2. Casalicchio E, Perciballi V (2019) Auto-scaling of containers: the impact of relative and absolute metrics. Future Generation Computer Systems 95:18–29
3. DirectXMan12 (2024) Prometheus adapter for Kubernetes metrics APIs. <https://github.com/kubernetes-sigs/prometheus-adapter>, accessed: 2026-04-01
4. Fette I, Melnikov A (2011) RFC 6455: The WebSocket protocol. Internet Engineering Task Force (IETF)
5. Imdoukh M, Ahmad I, Alfaiakawi MG (2020) Machine learning-based auto-scaling for containerized applications. Neural Computing and Applications 32:9745–9760
6. Kubernetes Authors (2024) Horizontal pod autoscaling. <https://kubernetes.io/docs/tasks/>

- run-application/horizontal-pod-autoscale/,
accessed: 2026-04-01
7. Kubernetes SIG API Machinery (2024) Kubebuilder – SDK for building Kubernetes APIs. <https://book.kubebuilder.io/>, accessed: 2026-04-01
 8. Kubernetes SIG Testing (2024) kind – kubernetes in docker. <https://kind.sigs.k8s.io/>, accessed: 2026-04-01
 9. Llorido-Bostrán T, Miguel-Alonso J, Lozano JA (2014) Auto-scaling techniques for elastic applications in cloud environments. *Journal of Grid Computing* 12(4):559–592
 10. Qu C, Calheiros RN, Buyya R (2018) Auto-scaling web applications in clouds: A survey and taxonomy. *ACM Computing Surveys* 51(4):1–33
 11. Rossi F, Nardelli M, Cardellini V (2020) Horizontal and vertical scaling of container-based applications using reinforcement learning. *Future Generation Computer Systems* 109:125–138
 12. Toka L, Dobreff G, Fodor B, Sonkoly B (2021) Machine learning-based scaling management for Kubernetes edge clusters. In: 2021 IEEE Conference on Network Function Virtualization and Software Defined Networks, IEEE, pp 1–7
 13. Verma A, Pedrosa L, Korupolu M, Oppenheimer D, Tune E, Wilkes J (2015) Large-scale cluster management at Google with Borg. In: *Proceedings of the Tenth European Conference on Computer Systems*, pp 1–17
 14. Yin L, Luo J, Luo H (2021) Horizontal pod autoscaling in Kubernetes for elastic container orchestration. *Sensors* 21(13):4539
 15. Zhong Z, Buyya R (2020) Machine learning-based orchestration of containers: A taxonomy and future directions. vol 54, pp 1–35